

3장

Stacks과 Queues

3.1 C++의 Templates	48
3.2 Adaptive containers: Stack과 Queue	50
3.3 Bag 자료구조	52
3.4 Stack 자료구조	56
3.5 Queue 자료구조	58
3.6 Deque 자료구조	62
3.7 미로 찾기	62
3.8 Stack을 이용한 Postfix expression	71
3.9 Queen 문제와 Knight tour 문제	76



3장

Stacks과 Queues

3.1 C++의 Templates

C++의 STL(Standard Template Library)은 container, iterator, function objects, algorithms을 표현하는 template들을 제공한다. container는 배열, bag, set, list와 같이 여러 개의 값들을 표현하는 클래스로 표현된다. iterator는 container내의 객체들을 순차적으로 접근하는 객체로서 pointer의 generalization이다. function objects은 function pointer이며 function 이름이 pointer로 처리된다.

C++ STL의 vector는 순서가 있는 dynamic 배열로서 sequence 에서 insert, delete를 처리함에 따라 size가 동적으로 변한다. Vector elements는 연속(contiguous) 공간에 저장되므로 iterator를 사용하여 접근 또는 traverse가 처리된다. Sequence의 중간 또는 처음 부분에 삽입과 삭제는 $O(n)$ 이 소요된다. Last element의 삭제는 $O(1)$ 이나 sequence 뒤끝에 insert는 size를 증가시키는 경우에는 복사하여 이동해야 하기 때문에 성능이 떨어진다.

vector template class는 sequence container로서 모든 element가 순차적으로 저장되는 데 배열과 다른 점은 메모리가 동적으로 할당되어 vector의 크기가 동적으로 변경된다. vector template class 정의에서 `allocator<T>[16]`는 메모리 할당과 반환으로 new, delete보다 좀 더 유연한 방식, 즉 fine-grained 방식으로 메모리를 관리하기 위해 사용한다. 기존 new 연산자는 메모리 할당, 값의 초기화를 SW 개발자가 원하는 방향으로 customizing을 할 수가 없다. new와 delete 연산자는 메모리 할당과 객체 생성을 동시에 처리한다. 즉, `new T`는 메모리를 할당하고 T의 생성자를 호출합니다. 마찬가지로 `delete p`는 소멸자를 호출한 후 메모리를 해제한다. 반면에 allocator class를 사용하여 vector container의 메모리 할당 방식을 바꿀 수가 있다. `std::vector`는 기본적으로 `std::allocator`를 사용하여 메모리를 관리한다. 아래 예제는 `std::vector`를 선언할 때 두 번째 template 인자로 `std::allocator`를 명시적으로 사용하는 방법을 보여준다.

```
#include <iostream>
#include <vector>
```

```

#include <memory> // std::allocator가 정의된 헤더

int main() {
    // std::vector<int>는 기본적으로 std::vector<int, std::allocator<int>>와
    같다.
    // 여기서는 명시적으로 std::allocator를 사용했다.
    std::vector<int, std::allocator<int>> my_vector;

    // push_back을 호출하면 내부적으로 allocator의 allocate, construct가
    사용된다.
    my_vector.push_back(10);
    my_vector.push_back(20);
    my_vector.push_back(30);

    // 벡터의 요소를 순회하며 출력한다.
    for (const auto& val : my_vector) {
        std::cout << val << " ";
    }
    std::cout << std::endl; // 출력: 10 20 30

    // 벡터가 범위를 벗어나면 내부적으로 allocator의 destroy, deallocate가
    호출된다.
    return 0;
}

```

std::vector에 push_back()을 사용하여 요소를 추가할 때, 내부적으로 std::allocator는 다음과 같은 방식으로 동작한다. 메모리 부족 시 재할당 처리는 my_vector의 현재 용량(capacity)이 부족하면, std::allocator의 allocate() 함수를 호출하여 더 큰 메모리 공간을 할당받는다. 객체 복사는 기존의 요소들이 새로운 메모리 공간으로 이동(move)되거나 복사(copy)됩니다. 이때 std::allocator의 construct() 함수가 사용된다. 새 요소 추가는 push_back()으로 전달된 새 요소가 construct() 함수를 통해 새로운 메모리 공간에 생성된다. 기존 메모리 해제는 이전의 메모리 공간을 std::allocator의 deallocate()와 destroy() 함수를 통해 해제됩니다.

Container class는 memory를 효율적으로 관리할 수 있어야 하는데 new/delete는 메모리를 할당하고 모든 요소를 초기화하도록 구현되어 있다. 이러한 new/delete는 컴퓨터 자원의 사용량이 상당히 증가하게 된다. 그리고 new는 개발

자가 원하는 메모리 할당 방식으로 customizing을 할 수가 없다. 반면에 vector template은 allocator를 추가로 사용할 수 있다.

```
template <typename T, class Allocator = allocator<T>> class vector;
```

allocate()는 vector의 메모리 할당 시에 초기화되지 않는 메모리 공간을 할당하여 준다. 할당된 메모리에 대한 초기화를 해주지 않으므로 성능이 new보다 더 빠르다. deallocate()는 allocate()로 할당된 메모리 공간을 해제한다. construct()함수는 T*가 가리키는 pointer에 const T& 객체를 저장한다. destroy() 함수는 객체를 소멸시킨다.

```
template <class T> class allocator {
public:
    T* allocate(size_t); //T 객체의 개수가 size_t
    void deallocate(T*, size_t);
    void construct(T*, const T&);
    void destroy(T*);
    ..
};
```

vector 이외에 deque, list container에 대한 template class도 class allocator를 사용하여 정의할 수 있다. deque는 queue의 양쪽 끝에서 add, delete가 처리된다. STL의 deque, list를 class allocator로 구현하는 실습을 통해 직접 메모리 할당과 반환, 그리고 메모리 초기화 실습을 해본다. C++ 표준 컨테이너(vector, deque, list, map, ...)는 메모리 관리 방식을 일반화하기 위해 Allocator라는 템플릿 매개변수를 둔다. default 값은 항상 std::allocator<T>이다. 즉, 컨테이너는 내부적으로 new/delete를 직접 쓰지 않고, Allocator 객체를 통해 메모리를 할당/해제한다는 의미이다. std::allocator<T>는 C++ 표준에서 제공하는 가장 기본적인 할당자이다. T는 컨테이너에 저장할 원소 타입이다. allocator<T>는 T 타입의 원소를 위한 메모리 공간을 잡고, 생성/소멸을 관리하는 기능을 가진다.

```
template <class T, class Allocator = allocator<T>>
class deque;
```

```
template <class T, class Allocator = allocator<T>>
class list;
```

3.2 Adaptive containers: Stack과 Queue

Container adapter는 sequence container를 encapsulate하는 class template이다. C++ STL에서 container adapter[18]는 stack, queue, priority_queue가 있다. 기존 sequence container에 대하여 class template로 정의된 adapter의 interface로 다른 functions이 제공된다.

stack<T>은 adapter class template로서 deque<T> container 를 encapsulate 하며 last-in-first-out (Last-In-First-Out, LIFO)를 구현한다. 두 번째 template parameter인 Container는 class stack의 실제 구현 Container가 deque<T>임을 나타낸다.

```
template <class T, class Container = deque<T>>
class stack;
```

deque template class는 다음과 같이 구현한다.

```
template <class T, class Allocator = allocator<T>>
class deque;
```

template class stack의 구현을 deque가 아닌 vector<T>를 사용하고자 할 때는 두 번째 parameter로서 다른 container를 변수 선언 시에 사용하면 된다. stack의 구현을 vector<>로 바꿀 수 있기 때문에 container adapter라고 부른다.

```
stack<double, vector<double>>> dstack;
```

queue<T>는 adapter class template로서 deque<T> container를 encapsulate

한다. first-in-first-out (First-In-First-Out, FIFO)를 지원하는 queue가 container adapter이다.

```
template <class T, class Container = deque<T>>
class queue;
```

template class queue의 구현을 deque가 아닌 list<T>를 사용하고자 할 때는 두 번째 parameter로서 다른 container를 변수 선언 시에 사용하면 된다. queue의 구현을 list<>로 바꿀 수 있기 때문에 container adapter라고 부른다.

```
queue<double, list<double>> dqqueue;
```

priority_queue<T>는 adapter class template로서 vector<T> container 를 encapsulate한다. queue가 largest element가 queue의 front에 항상 오도록 priority를 기준으로 항상 sort하도록 구현한다.

```
template <class T, class Container = vector<T>, class Compare = less<T>>
class priority_queue;
```

template class priority_queue의 구현을 deque가 아닌 vector<T>를 사용하고 자 할 때는 두 번째 parameter로서 다른 container를 변수 선언 시에 사용하면 된다. priority_queue의 구현을 deque<>로 바꿀 수 있기 때문에 container adapter라고 부른다.

```
// priority_queue<int, deque<int>, greater<int>> pqqueue;

#include <iostream>
#include <queue>
#include <vector>

using namespace std;

int main() {
    priority_queue<int> pq; // 기본: vector<int>, less<int>
```

```

pq.push(10);
pq.push(30);
pq.push(20);

cout << "Priority queue (max-heap): ";
while (!pq.empty()) {
    cout << pq.top() << " "; // 가장 큰 값 출력
    pq.pop();
}
cout << endl;
return 0;
}

```

3.3 Bag 자료구조

container class는 여러 개의 data objects을 저장하는 data structure를 표현하는 class로서 bag, set, linked list, stack, queue를 말한다. 이러한 container class를 3.1절의 C++ STL을 사용하거나 직접 구현할 수 있다. 예를 들면 클래스 bag는 duplicate, no ordering을 지원하는 container 클래스이다. bag에 대한 Pop() 실행 시에 아무 element를 return하게 된다. bag에 Push() 실행 시에는 가방에 물건 넣듯이 저장 순서가 없고 이미 있는 물건도 또 넣을 수 있게 된다.

container class를 직접 구현할 때 bag의 behavior로서 no ordering과 duplicate를 허용하는 Push(), Pop() 함수를 구현하는 것이 필요하다. bag 자료구조는 class definition으로 구현할 때 1) 데이터 구조, 2) operation, 3) 제약조건이 표현되어야 한다. bag 데이터 구조는 class definition에서 주석으로 표현하는데 element의 collection이라는 타입으로 기술한다. bag의 operation은 class의 public function으로 기술한다. bag의 제약조건은 class의 private function으로 기술하여 empty일때와 full일 때를 check한다. Bag<>에 capacity 보다 많은 데이터를 Push()할 때 용량을 2배로 늘리기 위한 ChangeSizeID()를 호출한다. 기존에 Push()된 데이터는 memcpy()로 복사한다.

Bag[1]의 Push() 구현은 element를 array의 first available position에 저장하며

bag에 저장되어 있는 element에 대한 중복 검사를 하지 않는다. bag 내부에서 element의 순서가 없는 것은 Pop() 구현으로 처리한다. Pop()은 array의 middle position의 element를 삭제하고 return하여 순서가 없는 것을 처리한다. 배열의 middle이 아닌 임의 장소의 객체를 삭제하는 것으로도 구현이 가능하다. 중복이 있고 순서가 없는 bag 자료구조는 bag::Push()와 bag:: Pop()의 함수 body로 구현된다. Pop()의 완전 구현을 구해 memcpy() 함수 호출을 수정하는 것이 필요하다.

```
//소스 코드3.1: Bag 자료구조
#pragma once
#include <iostream>
#include <memory>
using namespace std;
#define MaxSize 100

template <class T>
class Bag {
public:
    Bag(int bagCapacity = 10);
    ~Bag( );
    bool IsFull( );
    int Size( ) const;
    bool IsEmpty( ) const;
    virtual T& Pop( ) const;
    virtual void Push(const T&);
private:
    T* array;
    int capacity;
    int top;
};

//Visual Studio2019에서 bag.h, bag.cpp로 분리하지 않아야 함
template <class T>
Bag<T>::Bag(int bagCapacity) : capacity(bagCapacity)
{
    if (capacity < 1) throw "Capacity must be > 0";
    array = new T[capacity];
    top = -1;
}

template <class T>
Bag<T>::~Bag( ) { delete[ ] array; }
```

```

template <class T>
void ChangeSizeID(T*& a, const int oldSize, const int newSize)
{
    if (newSize < 0) throw "New length must be >= 0";

    T* temp = new T[newSize];
    int number = oldSize;
    if (oldSize > newSize) number = newSize;
    //copy(a, a + number, temp);
    memcpy(temp, a, number);
    delete[ ] a;
    a = temp;
}

template <class T>
int Bag<T>::Size( ) const {
    return top + 1;
}

template <class T>
bool Bag<T>::IsEmpty( ) const {
    return top < 0;
}

template <class T>
void Bag<T>::Push(const T& x)
{
    if (top == capacity - 1)
        // 현재 버전은 ordering 상태에서 push한다. non-ordering되게 push가 가능하게
수정
    {
        ChangeSizeID(array, capacity, 2 * capacity);
        capacity *= 2;
    }
    array[++top] = x;
}

template <class T>
T& Bag<T>::Pop( ) const
{
    T retValue;
    if (IsEmpty( )) throw "Bag is empty, cannot delete";
    int deletePos = top / 2;
    retValue = array[deletePos];
    // 실습 사항: no ordering상태로 pop되게 수정

```

```

        //copy(array + deletePos + 1, array + top + 1, array + deletePos);
        memcpy(array + deletePos, array + deletePos, top + 1);
        top--;
        return retValue;
    }

template <class T>
inline bool Bag<T>::IsFull( )
{
    if (top == MaxSize - 1) return TRUE;
    else return FALSE;
}

int main( ) {
    Bag<int> b(4);
    //Bag<float> fo(10);에 대하여도 구현할 것
    //화면에서 임의 숫자를 입력받아 Push( ) 호출하도록 수정
    b.Push(3);
    b.Push(5);
    b.Push(1);
    b.Push(6);
    //cin >> b; //가능하게 수정
    if (b.IsEmpty( )) {
        cout << "empty" << endl;
    }
    else {
        cout << "not empty" << endl;
    }

    while (!b.IsEmpty( )) {
        cout << b.Element( ) << endl;
        b.Pop( );
    }
    //cout << b; //가능하게 수정
    system("pause");
    return 0;
}

```

3.4 Stack 자료구조

stack은 bag와 달리 순서가 있는 sequence이며 $A = a_0, a_1, \dots, a_{n-1}$ 는 $n \geq 0$ elements의 ordered list이다. stack은 ordered list의 한쪽 끝에서 Push(),

Pop()이 일어나는 Last In First Out(LIFO)를 지원한다. 그리고 stack element의 data type은 template을 사용하여 instantiate 한다. Stack<>에 Push()할 때 capacity까지 가득 차면 용량을 2배로 늘리기 위해 ChangeSizeID()를 호출하고 기존 입력된 데이터는 memcpy()로 복사한다.

```
//소스 코드3.2: Stack 자료구조
#include <iostream>
#include <memory>

using namespace std;

template <class T>
class Stack {
public:
    Stack(int stackCapacity = 10);
    bool IsEmpty( ) const;
    T& Top( ) const;
    void Push(const T& item);
    void Pop( );

private:
    T* stack;
    int top;
    int capacity;
};

template <class T>
Stack<T>::Stack(int stackCapacity) :capacity(stackCapacity)
{
    if (capacity < 1) throw "Stack capacity must be > 0";
    stack = new T[capacity];
    top = -1;
}

template <class T>
void ChangeSizeID(T*& a, const int oldSize, const int newSize)
{
    if (newSize < 0) throw "New length must be >= 0";

    T* temp = new T[newSize];
    int number = oldSize;
    if (oldSize > newSize) number = newSize;
    //copy(a, a + number, temp);
}
```

```

        memcpy(temp, a, number);
        delete[ ] a;
        a = temp;
    }

template <class T>
inline bool Stack<T>::IsEmpty( ) const { return top == -1; }

template <class T>
inline T& Stack<T>::Top( ) const
{
    if (IsEmpty( )) throw "Stack is empty";
    return stack[top];
}

template <class T>
void Stack<T>::Push(const T& x)
{
    if (top == capacity - 1)
    {
        ChangeSizeID(stack, capacity, 2 * capacity);
        capacity *= 2;
    }
    stack[++top] = x;
}

template <class T>
void Stack<T>::Pop( )
{
    if (IsEmpty( )) throw "Stack is empty. Cannot delete.";
    stack[top--].~T( );
}

int main( ) {

    Stack<int> s(4);
    //Stack<float> fo(10); 실수에 대하여도 구현
    //화면에서 입력받아 동작하도록 구현
    //stack이 full일 때 처리 기능을 구현
    s.Push(3);
    s.Push(6);
    s.Push(1);
    s.Push(2);
    cin >> s;//구현 실습
    cout << "is empty: ";
    if (s.IsEmpty( )) {

```

```

        cout << " yes" << endl;
    }
    else {
        cout << " no" << endl;
    }
    cout << "print all elements: " << endl;
    while (!s.IsEmpty( )) {
        cout << s.Top( ) << " ";
        s.Pop( );
    }
    cout << s; //구현 실습
    cout << endl;
    system("pause");
    return 0;
}

```

자료구조 bag와 stack을 구현하는 class definition[1]을 보면 별 차이가 없다. 데이터 representation도 array를 사용하고 top 변수를 쓰는 것도 같다. 단지 차이가 나는 것은 Push (), Pop() 함수의 구현 body가 다를 뿐이다. set 자료구조는 순서가 없고 중복이 없다. bag 또는 stack 처럼 set에 대한 자료구조로서 class definition을 구현하고 set::Push(), set::Pop()을 호출하는 main() 프로그램을 작성하는 실습이 container class를 익히는 데 도움이 된다. set을 bag의 subclass로 구현하는 실습 코딩이 유익하다.

3.5 Queue 자료구조

queue는 ordered list이며 모든 insertions(push)이 한쪽 끝에서, 모든 deletions(pop)은 다른 반대편 끝 쪽에서 일어난다. Push()는 queue의 끝 쪽인 rear에서 처리되고 Pop()은 queue의 앞쪽인 front에서 처리되기 때문에 First-In-First-Out(FIFO)를 지원한다. queue의 class definition에는 FIFO에 대한 언급이 없다. queue 자료구조의 public function은 Push()와 Pop()으로 stack 자료구조와 같다. queue의 FIFO는 Push()와 Pop() 함수의 body에서 구현되고 있다. 따라서 queue::Push()와 queue::Pop()를 사용할 때에 이 두개의 함수가 FIFO를 구현하고 있다. 다시 말하면 queue의 FIFO semantics은 Push(), Pop() signature로 추상화된 것이다.

queue의 단순 구현(simple queue라고 부른다)[1]으로서 array를 사용할 때 capacity를 넘어서는 Push() 처리가 필요하다. simple queue의 element 개수는 capacity보다 적지만 rear가 array의 끝을 가리키게 되면 더 이상 Push() 처리가 안 된다. Push() 함수의 구현에서 (rear + 1) == capacity이면 용량을 2배로 늘리고 기존 입력된 데이터를 새로 할당된 배열에 복사하는 memcpy()함수를 수정해야 한다.

queue가 아직 비어 있는 공간이 많음에도 불구하고 (rear + 1 > capacity)이면 queue의 용량을 2배로 늘려야 하는 문제를 해결하기 위하여 array를 circular 방식으로 이용하는 것을 circular queue라고 부른다. circular queue는 simple queue의 class definition과 동일하며 Push(), Pop() 함수의 body만 다르게 (rear = (rear + 1) % capacity; front = (front + 1) % capacity;) 구현된다. 다음과 같은 circular queue의 source code를 고쳐 simple queue를 구현하는 실습 코딩으로 차이를 살펴보는 것이 도움이 된다. circular queue가 full 이면 용량을 2배로 늘리고 기존 데이터를 복사하는 memcpy() 함수 호출 구문을 수정해야 한다.

```
//소스 코드3.3: Queue 자료구조
#include <iostream>
#include <memory>

using namespace std;

template <class T>
class Queue
{
public:
    Queue(int queueCapacity = 10);
    bool IsEmpty( );
    T& Front( );
    T& Rear( );
    void Push(T const& x);
    void Pop( );
private:
    T* queue;
    int front;
    int rear;
    int capacity;
};

template <class T>
Queue<T>::Queue(int queueCapacity) : capacity(queueCapacity)
```

```

{
    queue = new T[capacity];
    front = rear = -1;
}

template <class T>
inline bool Queue<T>::IsEmpty( ) { return front == rear; }

template <class T>
inline T& Queue<T>::Front( )
{
    if (IsEmpty( )) throw "Queue is empty. No front element";
    return queue[(front + 1) % capacity];
}

template <class T>
inline T& Queue<T>::Rear( )
{
    if (IsEmpty( )) throw "Queue is empty. No rear element";
    return queue[rear];
}

// doubling queue capacity
template <class T>
void Queue<T>::Push(T const& x)
{
    if ((rear + 1) % capacity == front)
    {
        T* newQueue = new T[2 * capacity];
        // allocate an array with twice the capacity
        int start = (front + 1) % capacity;
        if (start < 2)
            //copy(queue + start, queue + start + capacity - 1, newQueue);
            memcpy(newQueue, queue + start, start + capacity - 1);
        else
        {
            //copy(queue + start, queue + capacity, newQueue);
            //copy(queue, queue + rear + 1, newQueue + capacity - start);
            memcpy(newQueue, queue + start, capacity);
            memcpy(newQueue + capacity - start, queue, rear + 1);
        }
        // switch to new Queue
        front = 2 * capacity - 1;
        rear = capacity - 2;
        capacity *= 2;
    }
}

```



```

        delete[ ] queue;
        queue = newQueue;
    }
    rear = (rear + 1) % capacity;
    queue[rear] = x;
}

template <class T>
void Queue<T>::Pop( )
{
    if (IsEmpty( )) throw "Queue is empty. Cannot delete.";
    front = (front + 1) % capacity;
    queue[front].~T( );
}

void main( )
{
    Queue<int> s(3);
    s.Push(2);
    s.Push(1);
    s.Push(4);
    //s.Push(10); //구현시에 doubling 동작 가능한지 테스트
    cin >> s; //구현 실습
    if (s.IsEmpty( )) {
        cout << "empty" << endl;
    }
    else {
        cout << "not empty" << endl;
    }
    while (!s.IsEmpty( )) {
        cout << s.Front( ) << endl;
        s.Pop( );
    }
    //cout << s << endl; //가능하도록 필요한 함수를 추가 구현한다
    system("pause");
}

```

3.6 Deque 자료구조

Deque(double-ended queue)는 양쪽 끝에서 삽입과 삭제가 모두 가능한 자료구조이다. Deque는 양쪽 끝에서 insert는 Push_front(), Push_rear()로 처리한다. Pop_front(), Pop_rear()도 구현한다. DeQueue의 용량이 full이면 ChangeSizeID()를

호출하여 용량을 2배 늘리는 구현 실습을 한다. 마지막으로 Circular DeQue을 구현하는 실습을 완성한다.

```
#include <deque>
using namespace std;
deque<string> dq;
deque<string> d(10); //0으로 초기화된 10개 element 생성
deque<string> d2(d); //복사 생성자
dq.front( ); //1st element
dq.back( ); //마지막 element
dq.push_front("a");
dq.push_back("b");
dq.pop_front( );
dq.pop_back( );
```



그림 3.1 Deque 구조.

```
template <class T>
class DeQue
{
public:
    DeQue(int queueCapacity = 10);
    bool IsEmpty();
    T& Front();
    T& Rear();
    void Push_front(T const& x);
    void Pop_front();
    void Push_rear(T const& x);
    void Pop_rear();
    friend ostream& operator<<<T>(ostream& os, DeQue<T>&);
    friend istream& operator>>>T>(istream& os, DeQue<T>&);
private:
    T* queue;
    int front;
    int rear;
    int capacity;
};
```

3.7 미로 찾기

미로에서 mouse가 입구에서 출발하여 출구로 나올 수 길을 찾는 문제[1]이다. 다음 행렬에서 mouse는 0이면 지나갈 수가 있고 1이면 지나가지 못한다. 그리고 행렬 밖으로는 나가지 못한다. main()에서 표현된 입력 데이터 `intinput[12][15]` 은 미로 행렬을 나타낸다. input 테이블을 상하 울타리, 좌우 울타리를 친 변형 테이블을 2차원 배열 `maze[i][j]`로 표현할 때 $1 \leq i < m$, $1 \leq j \leq p$ 이고 ($m = 14$, $p = 17$ 이 된다) blocked path 은 1이고 지나갈 수 있는 경로는 0이다. $i = 0$ 또는 $i = m-1$, $j = 0$ 또는 $j = p-1$ 은 원래 미로 밖이므로 나갈 수 없다. 변형된 미로 `maze[1][1]`에서 시작하며, exit는 `maze[m-2][p-2]`에서 종료된다.

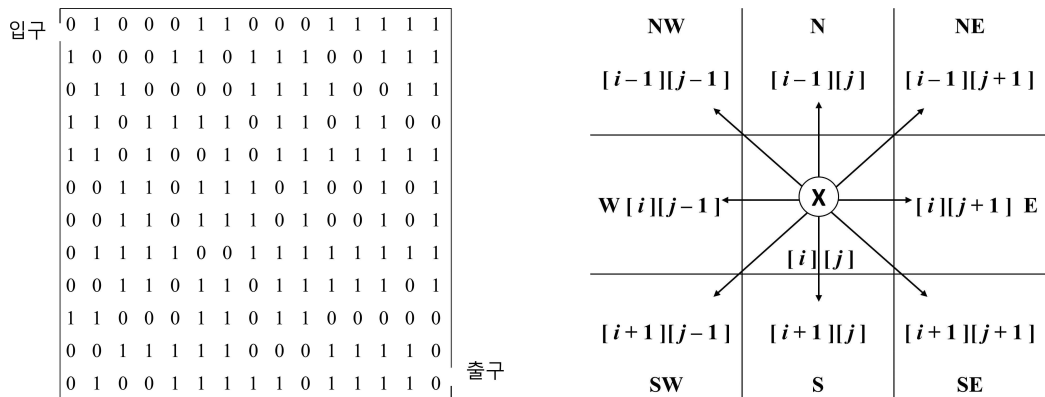


그림 3.2 미로 표현과 이동 방향.

mouse의 현재 위치가 $[i][j]$ 일 때 이동할 수 있는 경로는 다음과 같이 8가지가 있다. 8가지는 north, northeast, east, southeast, south, southwest, west, northwest이고 각각을 N, NE, E, SE, S, SW, W, NW로 표기한다. 0 ~ 7 사이의 이동 방향을 숫자 0 ~ 7 대신에 “enum directions {N, NE, E, SE, S, SW, W, NW};” 을 사용한다.

mouse의 현재 위치 `maze[i][j]`가 미로의 가장자리에 있을 때, 즉 $i = 1$ or $m-1$, $j = 1$ or $p-1$ 일 때는 가장자리이므로 밖으로 이동할 수가 없다.

이러한 border를 검사하는 if 문 사용을 줄이기 위해(소스 코드의 line의 숫자를 줄이는 것이 가장 잘하는 코딩이다) 확장된 미로를 maze[14][17]으로 선언한다. border를 모두 1로 설정하기 위하여 $i = 0$ or $m-1$, $j = 0$ or $p-1$ 에 대하여 “m[i][j] = 1;” 를 초기화한다.

현재 위치에서 이동 가능한 8가지 방향은 “enum directions q;” 로 선언되고 0 ~ 7의 값을 갖게 된다. mouse의 현재 위치가 maze[i][j] 라면 이동후의 위치 maze[g][h]는 다음과 같은 문장으로 표현된다.

```
struct offsets {
    int a, b;
}
offsets move[8];
//q는 0 ~ 7 값을 가짐
g = i + move[q].a; h = j + move[q].b;
```

q의 값에 따라 다음 이동 위치 maze[g][h]는 “offsets move[8];” 로 설정된 값을 사용하여 변경된다. 배열 move[q]의 값을 사용하여 q의 값으로 다음 위치를 이동한다. mouse가 막다른 길에 도달하면 stack을 이용하여 왔던 길로 되돌아가 가보지 않은 다른 경로를 찾아보아야 한다. 이때 한번 지나간 경로를 다시 선택하는 것을 없애기 위하여 또 다른 배열 mark[14][17]을 만들고 초기 값은 0으로 하고 지나가면 mark[i][j]을 1로 변경한다

q	$move[q].a$	$move[q].b$
N	-1	0
NE	-1	1
E	0	1
SE	1	1
S	1	0
SW	1	-1
W	0	-1
NW	-1	-1

그림 3.3 미로 벌레 이동 방향.

maze[] []를 사용한 미로 찾기 알고리즘은 현재 위치 maze[i][j]를 stack에 push한 후에 생성하여 다음 위치로 이동한다. 더 이상 경로가 없으면 stack에서 pop하여 되돌아가 가지 않은 다른 경로를 선택하여 미로 찾기를 계속한다. stack에 현재 위치를 push할 때 다음 이동 방향인 dir를 같이 포함한다. 미로 찾기 시작시에 (1,1,N)을 stack에 push한 후에 while() {}에서 pop한 결과를 temp 변수가 갖게 되고 d = temp.dir이 이동할 방향이 된다.

backtracking(물리기)한 경로는 0으로 mark하여 미로 이동 경로만 2로 표현한 결과를 만들어 내는 것이 정답이다. class items 객체를 stack에 push, pop하여 non-recursive 알고리즘으로 구현한다.

```
void path(int maze[][], int mark[][], int m, int p)//m = 12, p = 15
//출발점 (1,1), 이동 방향 dir = 2(2는 동쪽) 을 스택에 push
//initialize stack to the maze entrance coordinates and direction east;
while (stack is not empty)
{
    (i,j,dir) = coordinates and direction deleted from top of stack;
    현재 위치 (i,j)에 대하여 mark[] []을 1로 설정
    while (there are more moves)
    {
        (g,h) = coordinates of next move;
        if ((g == m) && (h == p)) {
            //success:
            (i,j)와 (g,h)에 대하여 mark 표시
            return;
        }
        if ((!maze[g][h]) //legal move
            && (!mark[g][h]) //haven't been here before
        {
            mark[g][h] = 1;
            dir = next direction to try;
            add (i,j,dir) to top of stack;
            i = g; j = h; dir = N;
        }
        else
            d++; //try next direction
    }
}
```

```

        (i,j) 현위치에 대한 mark를 취소
    }
    cout << "No path found" << endl;

```

```

struct items {
    int x, y, dir;
}

```

operator<<() 함수는 stack과 items에 대하여 overload되었다. operator<<이 stack의 private data member를 접근하기 위해서 class Stack의 friend로 선언되었다.

다음은 stack을 사용하여 미로찾기 알고리즘으로 C++로 구현한 코드는 소스코드 3.4와 같다.

//소스 코드3.4: Mazing Problem

```

/*
    1) 하나의 file로 구동해야 한다. 2) template를 사용해야 한다. 3) Stack is a
    Bag. 4. mouse가 지나간 궤적은 2로 출력한다. backtracking궤적은 0으로 복구해야 한
    다.
*/
#pragma once
#include <iostream>

#include <memory>
#include <fstream>

using namespace std;
#define MaxSize 100
const int DefaultSize = 100;

struct items {

```

```

        int x, y, dir;
};

template <class T>
class Bag {
public:
    Bag(int bagCapacity = 10);
    ~Bag();
    bool IsFull();
    int Size() const;
    bool IsEmpty() const;
    virtual T& Pop();
    virtual void Push(const T&);
    friend ostream& operator<<<T>(ostream& os, Bag<T>&);
    friend istream& operator>><T>(istream& os, Bag<T>&);
protected:
    T* array;
    int capacity;
    int top;
};

template <class T>
ostream& operator<<(ostream& os, Bag<T>& s) {

}

template <class T>
istream& operator>>(istream& os, Bag<T>& s) {

}

//Visual Studio2019에서 bag.h, bag.cpp로 분리하지 않아야 함
template <class T>
Bag<T>::Bag(int bagCapacity) : capacity(bagCapacity)
{
    if (capacity < 1) throw "Capacity must be > 0";
    array = new T[capacity];
    top = -1;
}

```

```
template <class T>
Bag<T>::~~Bag() { delete[] array; }
```

```
template <class T>
void ChangeSizeID(T*& a, const int oldSize, const int newSize)
{
    if (newSize < 0) throw "New length must be >= 0";

    T* temp = new T[newSize];
    int number = oldSize;
    if (oldSize > newSize) number = newSize;
    //copy(a, a + number, temp);
    memcpy(temp, a, number);
    delete[] a;
    a = temp;
}
```

```
template <class T>
int Bag<T>::Size() const {
    return top + 1;
}
```

```
template <class T>
bool Bag<T>::IsEmpty() const {
    return top < 0;
}
```

```
template <class T>
void Bag<T>::Push(const T& x)
{
    if (top == capacity - 1)
        // 현재 버전은 ordering 상태에서 push한다. non-ordering되게
        push가 가능하게 수정
    {
        ChangeSizeID(array, capacity, 2 * capacity);
        capacity *= 2;
    }
}
```



```

        array[++top] = x;
    }

template <class T>
T& Bag<T>::Pop()
{
    T retValue;
    if (IsEmpty()) throw "Bag is empty, cannot delete";
    int deletePos = top / 2;
    retValue = array[deletePos];
    // 실습 사항: no ordering상태로 pop되게 수정
    //copy(array + deletePos + 1, array + top + 1, array + deletePos);
    memcpy(array + deletePos, array + deletePos, top + 1);
    top--;
    return retValue;
}

template <class T>
inline bool Bag<T>::IsFull()
{
    if (top == MaxSize - 1) return true;
    else return false;
}

template <class T>
class Stack : public Bag<T>
{
public:
    Stack(int MaxStackSize = DefaultSize);
    T& Pop();
    void Push(const T&);
    //friend를 template에 사용시 사용법
    template <class T>
    friend ostream& operator<<(ostream& os, const Stack<T>& s);
    template <class T>
    friend ostream& operator<<(ostream& os, items& item);

```

```

private:
    void StackEmpty() { cout << "Stack is empty" << endl; };
    void StackFull() { cout << "Stack is full" << endl; };
};
//Visual Studio2019에서 bag.h, bag.cpp로 분리하지 않아야 함
template <class T>
Stack<T>::Stack(int stackCapacity) : Bag<T>(stackCapacity){}

template <class T>
T& Stack<T>::Pop()
{
    if (IsEmpty()) {
        StackEmpty();
    }
    T retValue;
    retValue = array[top--];
    return retValue;
}
template <class T>
void Stack<T>::Push(const T& t)
{
    if (IsFull()) StackFull();
    Bag<T>::Push(t);
}

template <class T>
ostream& operator<<(ostream& os, const Stack<T>& s)
{
    os << "top = " << s.top << endl;
    os << "stack elements: from the bottom to the top: " << endl;
    for (int i = 0; i <= s.top; i++)
        os << i << ":" << s.array[i] << endl;
    return os;
}

ostream& operator<<(ostream& os, items& item)
{

```

```

        return os << item.x << "," << item.y << "," << item.dir;
    }

    struct offsets {
        int a, b;
    };

    enum directions { N, NE, E, SE, S, SW, W, NW };
    offsets moves[8];
    int maze[100][100];
    int mark[100][100];

    void path(int m, int p)
    // Output a path (if any) in the maze: rows is m, cols is p:
    {
        //start at (1,1)
        mark[1][1] = 1;
        Stack<items> stack(m * p);
        items temp;
        temp.x = 1; temp.y = 1; temp.dir = E;
        stack.Push(temp);

        while (!stack.IsEmpty()) // stack not empty
        {
            temp = stack.Pop(); // unstack
            int i = temp.x; int j = temp.y; int d = temp.dir;
            mark[i][j] = 0; maze[i][j] = 0;
            while (d < 8) // moves forward
            {
                int g = i + moves[d].a;
                int h = j + moves[d].b;
                if ((g == m) && (h == p)) { // reached exit

                    // output path
                    mark[g][h] = 2; mark[i][j] = 2;
                    cout << "the term near the exit: " << i << " "
<< j << endl;

```

```

        cout << "exit: " << m << " " << p << endl;
        maze[i][j] = 2; maze[g][h] = 2;
        return;
    }
    if ((maze[g][h] == 0) && (mark[g][h] == 0)) { // new
position
        mark[g][h] = 1;
        //push the old temp to the stack, but the
direction changes.
        //Because the neighbor in the direction of d
has been checked.
        temp.x = i; temp.y = j; temp.dir = d + 1;
        stack.Push(temp); // stack it
        mark[i][j] = 2; maze[i][j] = 2;
        i = g; j = h; d = N; // moves to (g,h)
    }
    else d++; // try next direction
}
mark[i][j] = 0; maze[i][j] = 0;
}
cout << "no path in maze " << endl;
}
/*
maze[], mark[]를 class의 private data member로 선언
main()에서는 class의 public function의 호출로 미로 찾기
*/
void showMatrix(int d[][100], int row, int col) {
    for (int i = 0; i <= row; i++) {
        for (int j = 0; j <= col; j++) {
            cout << d[i][j] << " ";
        }
        cout << endl;
    }
}
void main() {
    int input[12][15] = {

```

```

{ 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1 },
{ 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1 },
{ 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1 },
{ 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0 },
{ 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1 },
{ 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1 },
{ 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1 },
{ 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1 },
{ 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1 },
{ 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0 },
{ 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0 },
{ 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0 },
};

```

```

moves[N].a = -1; moves[N].b = 0;
moves[NE].a = -1; moves[NE].b = 1;
moves[E].a = 0; moves[E].b = 1;
moves[SE].a = 1; moves[SE].b = 1;
moves[S].a = 1; moves[S].b = 0;
moves[SW].a = 1; moves[SW].b = -1;
moves[W].a = 0; moves[W].b = -1;
moves[NW].a = -1; moves[NW].b = -1;

```

```

for (int i = 0; i < 14; i++) {
    for (int j = 0; j < 17; j++)
    {
        if ((!i) || (!j) || (i == 13) || (j == 16))
            maze[i][j] = 1;
        else {
            maze[i][j] = input[i - 1][j - 1];
        };
        mark[i][j] = 0;
    }
}
showMatrix(maze, 13, 16);

```

```

    path(12, 15);
    showMatrix(mark, 13, 16);
    showMatrix(maze, 13, 16);
    int end;
    cin >> end;
}

```

3.8 Stack을 이용한 Postfix expression

C 언어에서 “A / B - C + D * E - A * C”를 expression이라 하며 evaluation 순서를 결정하기 위해 각 operator(+, -, *, / 등)에 대하여 연산자 우선순위(priority)를 부여한다. +, - 연산자보다 *, / 연산자가 먼저 실행되는 이유는 연산자 우선순위를 expression을 표현한 parse tree(expression을 표현한 syntax tree라고도 하며 구문을 나타낸다)에서 *, / 연산자가 +, - 연산자보다 밑에 있기 때문이다.

expression이 A*B/C일 때를 infix 표기라 하며 operators가 operands 사이에 온다. postfix form은 operator가 operands 다음에 온다 infix 표현식 A*B/C는 postfix 표현식으로 바꾸면 AB*C/이 된다. prefix는 operator가 먼저 온 후에 operand가 오는 것을 말한다. infix, prefix, postfix 표현식은 5장의 treed에 대한 inorder, preorder, postorder traversal 알고리즘의 결과와 같다.

먼저 stack을 사용하여 infix를 postfix 표기로 바꾸는 알고리즘을 구현한다. 그리고 stack을 사용하여 postfix form을 operand에 대하여 evaluate하는 소스 코드를 만든다. 좀 더 복잡한 infix expression을 postfix로 변환하는 알고리즘을 구현한다.

```

infix: A / B - C + D * E - A * C
postfix: A B / C - D E * + A C * -

```

infix를 postfix로 바꾸는 방법으로 stack을 이용한다. expression의 각 변수와 operand를 token으로 만든다. expression의 문자열 분석기를 lexical analyzer라 하며 operand로서 변수 또는 상수, operator를 추출한 것을 token이라 한다.

예를 들면 `sum = A + data;`라고 코딩할 때 token은 `sum, =, A, +, data`가 추출된다. 따라서 상수값, 변수명, `if`, `while` 등의 키워드가 하나의 token에 해당된다. 이러한 토큰은 lexical analyzer가 만들어 낸다.

stacking에 의해서 postfix를 만드는 방법은 expression의 token에 대하여 operand가 오면 바로 output로 보내고 operator는 stack에 push한다. infix 표기가 `A + B * C`일 때 stack에 있는 `+`에 대하여 다음 token이 `*`일 때 `*`의 priority가 `+` 보다 높기 때문에 `*`이 stack에 push된다. `A * B + C` 일때는 stack에 `*`이 있고 다음 token이 `+`이므로 stack에 있는 `*`이 pop된 후에 `+`가 stack에 push된다. input expression의 다음 token이 empty이면 stack에 남아 있는 모든 operators를 pop을 하여 마무리한다.

괄호가 있는 expression 은 우선순위가 높다. infix: `A * (B + C) * D`에 대하여 stack에 `*`을 push한 후에 `(` 는 `*` 보다 우선순위가 높기 때문에 stack에 들어가게 된다. 다음에 오는 `+`에 대하여 stack의 `(` 가 우선 순위가 낮기 때문에 `+` 를 stack에 넣게 된다. `)` 가 오게 되면 stack에서 `(` 이 나올 때까지 pop을 한다. postfix 결과는 `ABC+*D*`이 된다.

priority	operator
1	unary minus, !
2	*, /, %
3	+, -
4	<, <=, >=, >
5	==, !=
6	&&
7	

그림 3.4 연산자 우선순위.

operator의 stacking과 unstacking에 관한 priority-based scheme[1]이 사용된다. operators에 관한 2가지 priorities 규칙으로 isp(in-stack priority), icp(in-coming priority) 함수를 사용한다. 괄호에 대하여 isp('`(`') = 8, icp('`(`') = 0을 정의하고 '`)`'은 따로 정하지 않는다. token으로 '`)`'을 만나면 stack에서 '`(`'이 나올 때까지 pop을 하기 때문이다. stack의 최초 상태는 '`#`'을 먼저 push하므로 isp('`#`') = 8로 한다.

stacking rule은 operators의 in-stack priority가 in-coming priority의 숫자보다 작거나 같으면 pop (in-stack priority가 더 높다는 의미)을 한다. x가 다음 token이고 isp(stack>top())와 icp(x)비교하여 Push(x)를 Pop()을 처리한다.

```

void Eval(Expression e) {
    //e는 마지막 토큰이 '#'이라고 가정
    Stack<Token> stack;
    for (Token x=NextToken(e); x!='#'; x= NextToken(e))
        if (x is an operand)
            stack.Push(x)
        else {
            remove the correct number of operands for operator x from stack;
            perform the operation x and store the result (if any) onto the
            stack;
        }
    }

void Postfix(Expression e)
{
    Stack<Token> stack;
    stack.Push('#');
    for (Token x= NextToken(e); x!='#'; x=NextToken(e))
        if (x is an operand) cout<<x;
        else if (x == ')')
            { //unstack until '('
                for (;stack.Top() != '('; stack.Pop())
                    cout<<stack.Top();
            }
        else { //x is an operator
            for (; isp(stack.Top()) <= icp(x); stack.Pop())
                cout<<stack.Top();
            stack.Push(x);
        }
    // empty the stack
    for (;!stack.IsEmpty(); cout<<stack.Top(), stack.Pop());
    cout<<endl;
}

```



```
//소스 코드3.5: Postfix Expression
```

```
//token의 변수가 char 1개를 문자 숫자로 된 일반 변수가 처리되도록 구현  
//operator도 >=, && 연산자가 가능하도록 확장 구현
```

```
#include <iostream>  
#include <memory>  
#include <time.h>  
  
using namespace std;  
#define DefaultSize 100  
#define MaxSize 100  
template <class T>  
class Bag {  
public:  
    Bag(int bagCapacity = 10);  
    ~Bag();  
    bool IsFull();  
    int Size() const;  
    bool IsEmpty() const;  
    virtual T& Pop();  
    virtual void Push(const T&);  
protected:  
    T* array;  
    int capacity;  
    int top;  
};  
//Visual Studio2019에서 bag.h, bag.cpp로 분리하지 않아야 함  
template <class T>  
Bag<T>::Bag(int bagCapacity) : capacity(bagCapacity)  
{  
    if (capacity < 1) throw "Capacity must be > 0";  
    array = new T[capacity];  
    top = -1;  
}
```

```
template <class T>
Bag<T>::~~Bag() { delete[] array; }
```

```
template <class T>
void ChangeSizeID(T*& a, const int oldSize, const int newSize)
{
    if (newSize < 0) throw "New length must be >= 0";

    T* temp = new T[newSize];
    int number = oldSize;
    if (oldSize > newSize) number = newSize;
    //copy(a, a + number, temp);
    memcpy(temp, a, number);
    delete[] a;
    a = temp;
}
```

```
template <class T>
int Bag<T>::Size() const {
    return top + 1;
}
```

```
template <class T>
bool Bag<T>::IsEmpty() const {
    return top < 0;
}
```

```
template <class T>
void Bag<T>::Push(const T& x)
{
    if (top == capacity - 1)
        // 현재 버전은 ordering 상태에서 push한다. non-ordering되게
        push가 가능하게 수정
    {
        ChangeSizeID(array, capacity, 2 * capacity);
        capacity *= 2;
    }
}
```

```

        array[++top] = x;
    }

template <class T>
T& Bag<T>::Pop() {
    T retValue;
    if (IsEmpty()) throw "Bag is empty, cannot delete";
    int deletePos = top / 2;
    retValue = array[deletePos];
    // 실습 사항: no ordering상태로 pop되게 수정
    //copy(array + deletePos + 1, array + top + 1, array + deletePos);
    memcpy(array + deletePos, array + deletePos, top + 1);
    top--;
    return retValue;
}

template <class T>
inline bool Bag<T>::IsFull()
{
    if (top == MaxSize - 1) return true;
    else return false;
}

template <class T>
class Stack : public Bag<T>
{
public:
    Stack(int MaxStackSize = DefaultSize);
    T& Pop();
    void Push(const T&);
    T& Top() const;
    //friend를 template에 사용시 사용법
    template <class T>
    friend ostream& operator<<(ostream& os, const Stack<T>& s);
private:
    void StackEmpty() { cout << "Stack is empty" << endl; };

```

```

        void StackFull() { cout << "Stack is full" << endl; };
};
//Visual Studio2019에서 bag.h, bag.cpp로 분리하지 않아야 함
template <class T>
Stack<T>::Stack(int stackCapacity) : Bag<T>(stackCapacity) {
    if (capacity < 1) cout << "Stack capacity must be > 0";
}
template <class T>
T& Stack<T>::Top() const {
    return array[top];
}
template <class T>
T& Stack<T>::Pop()
{
    T retValue;
    if (IsEmpty()) StackEmpty();
    retValue = array[top--];
    return retValue;
}
template <class T>
void Stack<T>::Push(const T& t)
{
    if (IsFull()) StackFull();
    Bag<T>::Push(t);
}

template <class T>
ostream& operator<<(ostream& os, const Stack<T>& s)
{
    os << "top = " << s.top << endl;
    os << "stack elements: from the bottom to the top: " << endl;
    for (int i = 0; i <= s.top; i++)
        os << i << ":" << s.stack[i] << endl;
    return os;
}

typedef char Expression[20];

```

```

typedef char Token;
using namespace std;

int index = 0;
int inputData[26] = { 0 };
char NextToken(char* e)
{
    char token = e[index];
    index++;
    return token;
}

bool isOperand(char x)
{
    if (x == '+' || x == '-' || x == '*' || x == '/' ||
        x == '(' || x == ')' || x == '%' || x == '#')
        return false;
    else
        return true;
}

int isp(char a)
{
    if (a == '#')
        return 8; //가장 낮은 우선 순위
    else if (a == '!')
        return 1;
    else if (a == '*' || a == '/' || a == '%')
        return 2;
    else if (a == '+' || a == '-')
        return 3;
    else if (a == '<' || a == '>')
        return 4;
    else if (a == '(')
        return 8;
    return 0;
}

```

```

int icp(char a)
{
    if (a == '(')
        return 0;
    else if (a == '!')//unary minus를 추가한다
        return 1;
    else if (a == '*' || a == '/' || a == '%')
        return 2;
    else if (a == '+' || a == '-')
        return 3;
    else if (a == '<' || a == '>')// <=, >=를 추가한다
        return 4;
    // ==, != :: 5
    // && :: 6
    // || :: 7
    return 0;
}

void stringAdd(Expression& ex, char ch) {
    int len;
    len = strlen(ex);
    ex[len] = ch;
    ex[len + 1] = '\0';
}

//prefix() 알고리즘 구현
Expression& Postfix(Expression& e)
/*
화면에서 입력된 infix expression e로 부터 postfix를 구한다
NextToken()함수는 다음 토큰을 가져오는 함수이고 stack<>를 사용한다
infix expression의 마지막은 null 문자이고 생성된 postfix expression의 마지막 문자는
#이다.
stack의 bottom에 입력된 #를 사용하여 isp(), icp() 함수값의 규칙을 정의한다.
*/
{
    Stack<char> stack(30);
    stack.Push('#');

```

```

static char exp[20];
for (char x = NextToken(e); x != NULL; x = NextToken(e))
{
    char y;
    if (isOperand(x)) {
        stringAdd(exp, x);
    }
    else if (x == ')')
    {
        while (stack.Top() != '(')
        {
            x = stack.Top(); stringAdd(exp, x);
            stack.Pop();
        }
        stack.Pop();
    }
    else
    {
        while(isp(stack.Top()) <= icp(x)) {
            y = stack.Top(); stringAdd(exp, y);
            stack.Pop();
        }
        stack.Push(x);
    }
}

char z;
while ((z = stack.Top()) != '#')
{
    stringAdd(exp, z);
    stack.Pop();
}
stringAdd(exp, '#');
return exp;
}

void Eval(Expression e) {

```

```

/*
postfix expression e를 evaluate한다.
토큰은 operator, operand 또는 #이고 마지막 토큰은 #이다.
함수 NextToken()이 e로부터 다음 토큰을 가져온다.
변수 값은 난수로 생성된 inputData[] 배열 값을 사용하여 가져 온다.
*/
index = 0;
Stack<int> stack(30);
stack.Push(99);
int result = 0;
cout << "EVAL 시작::" << endl;
for (Token x = NextToken(e); x != '#'; x = NextToken(e))
    if (isOperand(x)) {
        int num = inputData[x - 'a']; cout << "변수값:: " <<
num;
        stack.Push(num);
    }
    else {
        //remove the correct number of operands for operator
x from stack;
        //perform the operation x and store the result onto
the stack;

        int num1 = stack.Pop();
        int num2 = stack.Pop();
        if (x == '*')
            result = num1 * num2;
        else if (x == '+')
            result = num1 + num2;
        //연산자 >=, && 등이 구현되도록 확장 구현
        stack.Push(result);

    }
result = stack.Pop();
cout << "Eval 값은 " << result << endl;
}

void main()

```



```

{
    srand(time(NULL));
    for (int i = 0; i < 26; i++)
        inputData[i] = rand() % 9;
    index = 0;
    char input[20];
    char* postfix;
    char ch;
    cout << endl << "infix 표기의 expression 입력: ";
    scanf("%s", input);
    cout << input << endl;
    postfix = Postfix(input);
    cout << postfix << endl;
    Eval(postfix);
    cin >> ch;
}

```

3.9 Queen 문제와 Knight tour 문제

8 queen 문제는 8x8 chess 판에 8개의 queen을 배치하는 문제이다. queen은 가로, 세로, 대각선에 다른 queen을 놓을 수가 없다. N=8일 때는 92개의 해가 있다. recursive 함수를 사용하지 않고 stack을 사용하여 구현한다. 체스판을 배열 data[8][8]로 표현하고 0으로 초기화한다. 각 행에 queen을 놓으면 data[x][y] = 1;하고 count를 증가시키고 현재 위치 (x,y)를 stack에 push한다. 그런 다음에 x를 증가하여 다음 행에 queen을 놓을 위치를 nextMove(int d[][COL], int row, int col)으로 구한다. 다음 행에 queen을 놓을 수가 없으면 stack에서 pop하여 (Point p = stack.Pop()) 물리기(backtracking)을 처리한다. stack에서 pop을 한다는 것은 queen 배치를 취소하는 것이므로 count를 감소하고, ix = p.getX(), iy = p.getY()로 구한 data[ix][iy] = 0으로 되돌리고 iy++를 처리하여 다음 컬럼에 대하여 queen을 놓을 수 있는지 검사한다.

stack을 사용한 8-queen 해를 non-recursive 알고리즘으로 구현하는 프로그래밍 스킬(skill)을 습득하는 것이 목표이다. 그림 3.5는 8개의 queen을 배치한 하나의 답이다. 92개의 답을 찾기 위해서는 stack에서 pop하여 다른 해가 있는지를

backtracking으로 해결한다.

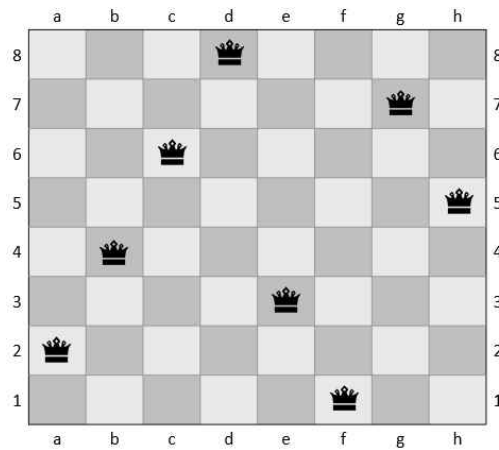


그림 3.5 8-Queen 문제.

//모든 해가 나오는 버전

/*

* 체스판은 8 x 8

* 체스의 기물: king/가로세로대각선 1칸만 이동, queen/가로세로 대각선/같은 편의 기물을 넘을 수 없다,

* Rook/가로,세로 이동/다른 기물을 넘을 수없다, bishop/대각선, knight/1-2칸 이동/다른 기물을 넘을 수 있다,

* pawn/처음 이동은 2칸까지 가능, 그 후 한칸만 가능, 잡을 때는 대각선 가능

* 체스판 최대 배치 문제 : king/16, Queen/8, rook/8, bishop/?, knight/?

* 체스 게임시에 rook 2개/a, h에 배치, knight 2개/b, g, bishop 2개/c, f, queen 1개/black queen은 black 칸에, pawn 8개를 배치한다

*/

#include <iostream>

#define ROW 8

#define COL 8

using namespace std;

class Point {

private:

int ix;

int iy;

public:

Point() { ix = 0; iy = 0; }

```

        Point(int x, int y) {
            ix = x;
            iy = y;
        }
        int getX() {
            return ix;
        }
        int getY() {
            return iy;
        }
    };

}

template <class T>
class Stack {
public:
    Stack(int stackCapacity = 10);
    bool IsEmpty() const;
    T& Top() const;
    void Push(const T& item);
    T& Pop();
    friend ostream& operator<<<T>(ostream& os, Stack<T>&);
    friend istream& operator>><T>(istream& os, Stack<T>&);
private:
    T* stack;
    int top;
    int capacity;
};

bool checkRow(int d[][COL], int crow) {
    //행 crow에 queen을 배치 가능한지 검사
}

bool checkCol(int d[][COL], int ccol) {
    //열 ccol에 queen을 배치 가능한지 검사
}

```

```

bool checkDiagSW(int d[][COL], int cx, int cy) { // x++, y-- or x--, y++ where
0<= x,y <= 7
//(cx,cy) 남서+북동 방향에 queen을 배치 가능한지 검사
}

```

```

bool checkDiagSE(int d[][COL], int cx, int cy) { // x++, y++ or x--, y--
//(cx,cy) 남동+북서 방향에 queen을 배치 가능한지 검사
}

```

```

bool checkMove(int d[][COL], int x, int y) { // (x,y)로 이동 가능한지를 check
//(x,y)에 queen을 배치 가능한지 검사
}
int nextMove(int d[][COL], int row, int col) { // 현재 row, col에 대하여 이동할 col을
return
//주어진 행 row의 col부터 시작하여 queen을 배치할 수 있는 첫 번째 col을 반환
//주어진 행 row의 col부터 나머지 열에 배치할 수 없으면 -1를 리턴
}

```

```

void showQueens(int data[][COL]) {
//체스판 결과를 출력
}
void solveQueen(int d[][COL]) { // 주어진 코드를 사용하여 완성
    Stack<Point>* st = new Stack<Point>(100);
    Point *p = new Point(ix, iy);
    st->Push(*p);
    while (true) {
        if (st->IsEmpty() && ix == 8)
            break;
        if ((iy = nextMove(d, ix, iy)) == -1) {
            if (st->IsEmpty())
                break;
            *px = st->Pop();
            continue;
        }
        px = new Point(ix, iy);
        st->Push(*px);
    }
}

```

```

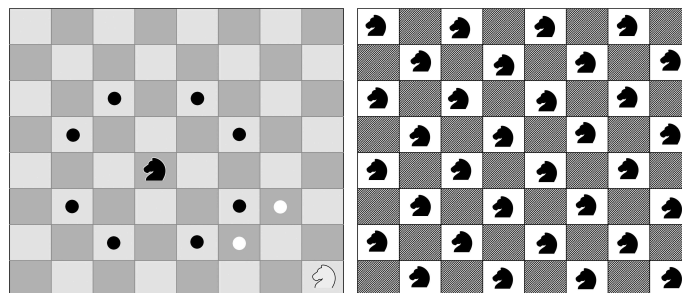
        if (count == 8) {
            showQueens(d);
            *px = st->Pop();
            continue;
        }
    }
}

void main() {
    const int row = ROW, col = COL;
    int data[row][col];
    for (int i = 0; i < row; i++)
        for (int j = 0; j < col; j++)
            data[i][j] = 0;

    solveQueen(data);
    system("pause");
    return;
}

```

chess 판의 nonattacking knight 문제는 8×8 chess 배열에 충돌되지 않는 knight 개수의 최솟값을 구하는 문제이다. 새로운 knight가 이미 배치된 knight로 부터 공격받지 않으면 추가 배치할 수 있다. knight의 최소 개수를 찾고 배치 결과를 보여주는 알고리즘을 8-queen 문제를 수정하여 구현하는 실습을 수행한다.



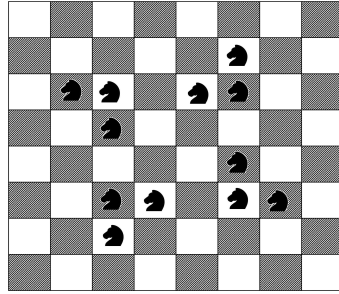


그림 3.6 Chess 판의 Knight 배치 해답.